VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors - Microcontrollers

Report lab 1

LED Animations

Advisor(s): Phan Van Sy
Student(s): Tran Le Dang Khoi 2452599

HO CHI MINH CITY, SEPTEMBER 2026

Contents

1 Exercise 1 **5**
1.1 Circuit Schematic 5
1.2 Source Code 5

2 Exercise 2 **6**
2.1 Requirement 6
2.2 Circuit Schematic 6
2.3 Source Code 7

3 Exercise 3 **8**
3.1 Requirement 8
3.2 Circuit Schematic 9
3.3 Source Code 9

4 Exercise 4 **11**
4.1 Requirement 11
4.2 Circuit Schematic 12
4.3 Source Code 14

5 Exercise 5 **15**
5.1 Requirement 15
5.2 Source Code 15

6 Exercise 6 **18**
6.1 Requirement 18
6.2 Circuit Schematic 19
6.3 Source Code 19

7 Exercise 7 **20**
7.1 Requirement 20
7.2 Source Code 20

8 Exercise 8 **21**
8.1 Requirement 21
8.2 Source Code 21

9 Exercise 9 **22**
9.1 Requirement 22
9.2 Source Code 22

10 Exercise 10 **22**
10.1 Requirement 22
10.2 Source Code 23

List of Figures

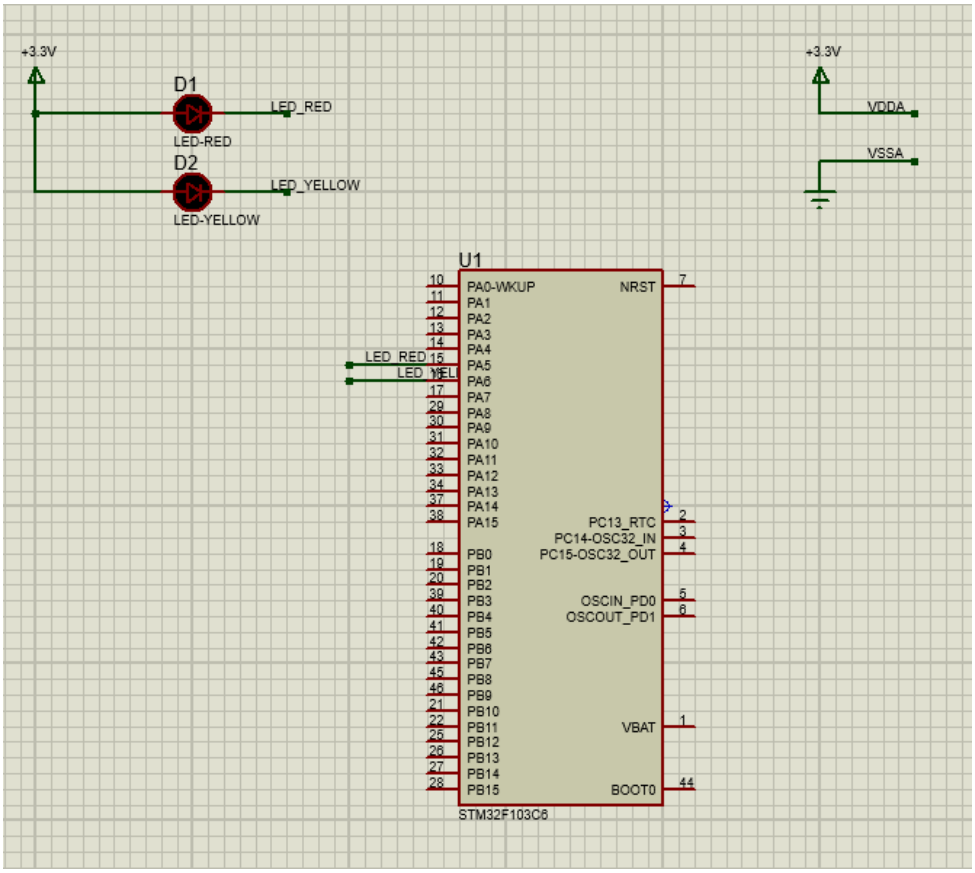
List of Tables

Listings

1 Exercise 1

1.1 Circuit Schematic

A second LED is connected to pin PA6 of the STM32F103C6. The red LED is connected to PA5, while the yellow LED is connected to PA6. Both LEDs are configured as active-low outputs.



Source here

1.2 Source Code

```
while (1)
{
  // Red ON, Yellow OFF
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
     GPIO_PIN_RESET);
```

```
5   HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_SET);
6   HAL_Delay(2000);
7
8   // Red OFF, Yellow ON
9   HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
       GPIO_PIN_SET);
10  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_RESET);
11  HAL_Delay(2000);
12 }
```

2 Exercise 2

2.1 Requirement

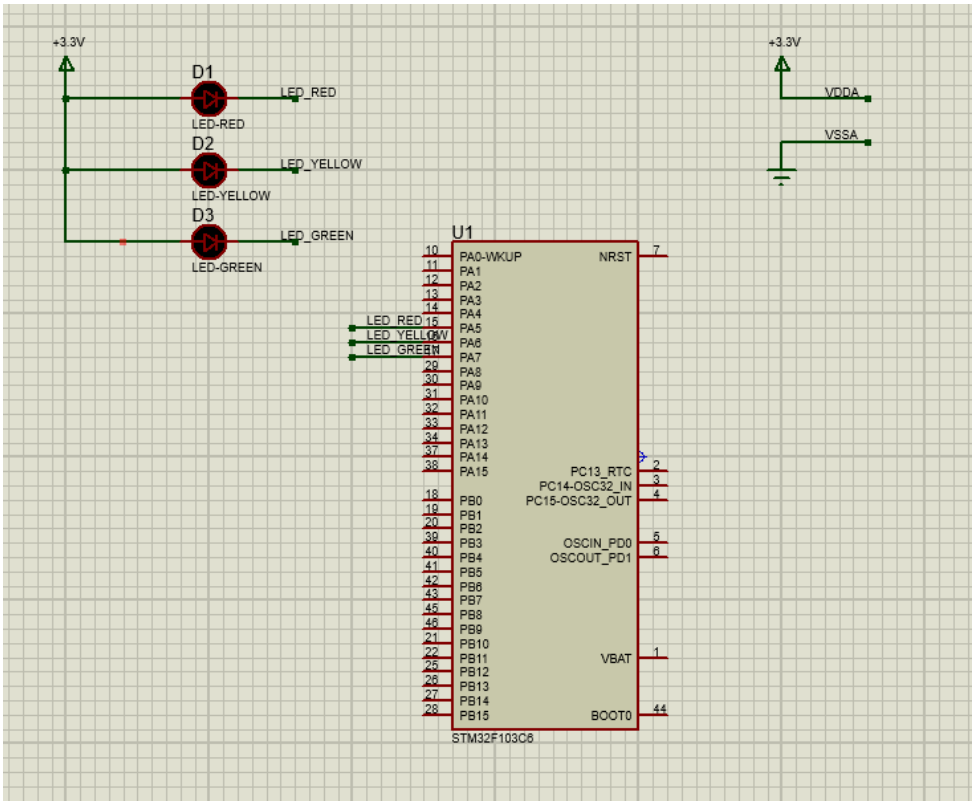
Exercise 2 extends the previous circuit by adding a green LED connected to pin PA7. The system simulates a traffic light with the following timing sequence:

- Red LED: 5 seconds.
- Yellow LED: 2 seconds.
- Green LED: 3 seconds.

The LEDs are connected in an active-low configuration. Therefore, a logic-low output turns an LED on and a logic-high output turns it off.

2.2 Circuit Schematic

Proteus schematic for the three-LED traffic light.



Source here

2.3 Source Code

```
while (1)
{
  // Red ON
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
     GPIO_PIN_RESET);
  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
     GPIO_PIN_SET);
  HAL_Delay(5000);

  // Yellow ON
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
     GPIO_PIN_SET);
```

```
11    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
          GPIO_PIN_RESET);
12    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
          GPIO_PIN_SET);
13    HAL_Delay(2000);
14
15    // Green ON
16    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
          GPIO_PIN_SET);
17    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
          GPIO_PIN_SET);
18    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
          GPIO_PIN_RESET);
19    HAL_Delay(3000);
20  }
```

3 Exercise 3

3.1 Requirement

Exercise 3 extends the previous traffic-light system to a four-way intersection. Twelve LEDs are arranged into four traffic lights, with each direction containing a red, yellow, and green LED.

Opposite directions operate simultaneously. Therefore, the North-South traffic lights share one set of control signals, while the East-West traffic lights share another set.

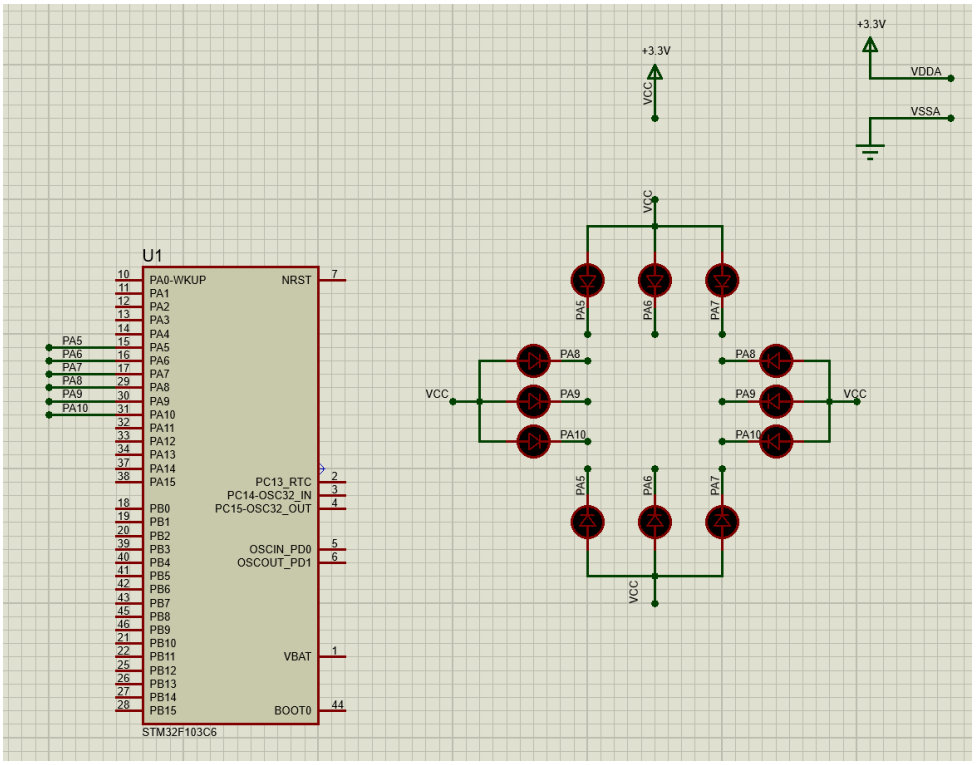
The traffic-light sequence uses the same timing as the previous exercise:

- Green LED: 3 seconds.
- Yellow LED: 2 seconds.
- Red LED: 5 seconds.

The LEDs are connected in an active-low configuration. Therefore, a logic-low output turns an LED on and a logic-high output turns it off.

3.2 Circuit Schematic

Proteus schematic for the four-way traffic-light system.



Source here

3.3 Source Code

```
while (1)
{
  // North-South GREEN, East-West RED
  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
     GPIO_PIN_RESET);

  HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
     GPIO_PIN_RESET);
```

```
9   HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
      GPIO_PIN_SET);
10  HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
      GPIO_PIN_SET);
11
12  HAL_Delay(3000);
13
14  // North-South YELLOW, East-West RED
15  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
      GPIO_PIN_SET);
16  HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
      GPIO_PIN_RESET);
17  HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
      GPIO_PIN_SET);
18
19  HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
      GPIO_PIN_RESET);
20  HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
      GPIO_PIN_SET);
21  HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
      GPIO_PIN_SET);
22
23  HAL_Delay(2000);
24
25  // North-South RED, East-West GREEN
26  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
      GPIO_PIN_RESET);
27  HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
      GPIO_PIN_SET);
28  HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
      GPIO_PIN_SET);
29
30  HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
      GPIO_PIN_SET);
```

```
31    HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
          GPIO_PIN_SET);
32    HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
          GPIO_PIN_RESET);
33
34    HAL_Delay(3000);
35
36    // North-South RED, East-West YELLOW
37    HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
          GPIO_PIN_RESET);
38    HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
          GPIO_PIN_SET);
39    HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
          GPIO_PIN_SET);
40
41    HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
          GPIO_PIN_SET);
42    HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
          GPIO_PIN_RESET);
43    HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
          GPIO_PIN_SET);
44
45    HAL_Delay(2000);
46  }
```

4 Exercise 4

4.1 Requirement

Exercise 4 extends the four-way traffic-light circuit by adding a single seven-segment display. The component used in Proteus is `7SEG-COM-ANODE`.

The common-anode pin is connected to the +3.3 V power supply, while segments a to g are connected to pins PB0 to PB6, respectively:

- Segment a: PB0.

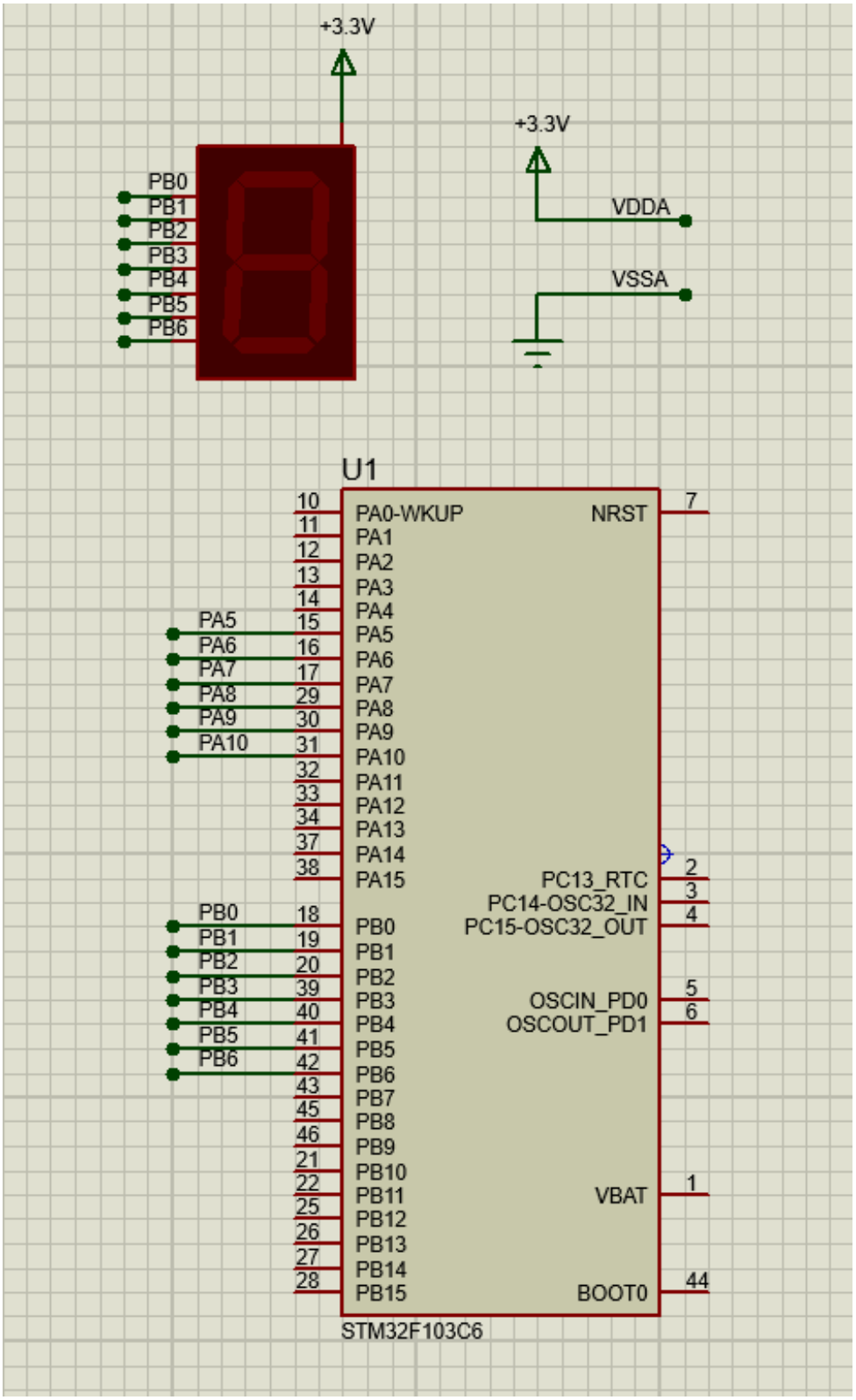
- Segment b: PB1.
- Segment c: PB2.
- Segment d: PB3.
- Segment e: PB4.
- Segment f: PB5.
- Segment g: PB6.

Since a common-anode seven-segment display is used, each segment is active-low. Therefore, a logic-low output turns a segment on, while a logic-high output turns it off.

The `display7SEG(int num)` function is implemented to display digits from 0 to 9. For testing, the displayed value is increased every second and returns to 0 after reaching 9.

4.2 Circuit Schematic

Proteus schematic for the four-way traffic light with a seven-segment display.


+3.3V
+3.3V
PB0
PB1
PB2
PB3
PB4
PB5
PB6
VDDA
VSSA
U1
PA0-WKUP
PA1
PA2
PA3
PA4
PA5
PA6
PA7
PA8
PA9
PA10
PA11
PA12
PA13
PA14
PA15
PB0
PB1
PB2
PB3
PB4
PB5
PB6
PB7
PB8
PB9
PB10
PB11
PB12
PB13
PB14
PB15
NRST
PC13_RTC
PC14-OSC32_IN
PC15-OSC32_OUT
OSCIN_PD0
OSCOUT_PD1
VBAT
BOOT0
STM32F103C6

Source here

4.3 Source Code

```
void display7SEG(int num)
{
    static const uint8_t seg_code[10][7] = {
        {0,0,0,0,0,0,1}, // 0
        {1,0,0,1,1,1,1}, // 1
        {0,0,1,0,0,1,0}, // 2
        {0,0,0,0,1,1,0}, // 3
        {1,0,0,1,1,0,0}, // 4
        {0,1,0,0,1,0,0}, // 5
        {0,1,0,0,0,0,0}, // 6
        {0,0,0,1,1,1,1}, // 7
        {0,0,0,0,0,0,0}, // 8
        {0,0,0,0,1,0,0}  // 9
    };

    static const uint16_t seg_pins[7] = {
        GPIO_PIN_0, // a
        GPIO_PIN_1, // b
        GPIO_PIN_2, // c
        GPIO_PIN_3, // d
        GPIO_PIN_4, // e
        GPIO_PIN_5, // f
        GPIO_PIN_6  // g
    };

    if (num < 0 || num > 9)
        return;

    for (int i = 0; i < 7; i++)
    {
        HAL_GPIO_WritePin(GPIOB, seg_pins[i], seg_code[num][i
           ] ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

5 Exercise 5

5.1 Requirement

Exercise 5 integrates the seven-segment display from the previous exercise into the four-way traffic-light system. The seven-segment display is used to show the remaining time of each traffic-light phase as a countdown.

The `display7SEG(int num)` function implemented in Exercise 4 is reused. A new `countdown(int seconds)` function is added to update the seven-segment display once every second.

The traffic-light sequence is:

- North-South green: 3 seconds.
- North-South yellow: 2 seconds.
- East-West green: 3 seconds.
- East-West yellow: 2 seconds.

During each phase, the opposite direction remains red.

5.2 Source Code

```
void countdown(int seconds)
{
  for (int i = seconds; i > 0; i--)
  {
    display7SEG(i);
    HAL_Delay(1000);
  }
}

while (1)
{
  // North-South GREEN, East-West RED
  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
     GPIO_PIN_SET);
```

```
14  HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
       GPIO_PIN_SET);
15  HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
       GPIO_PIN_RESET);
16
17  HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
       GPIO_PIN_RESET);
18  HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
       GPIO_PIN_SET);
19  HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
       GPIO_PIN_SET);
20
21  countdown(3);
22
23  // North-South YELLOW, East-West RED
24  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
       GPIO_PIN_SET);
25  HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
       GPIO_PIN_RESET);
26  HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
       GPIO_PIN_SET);
27
28  HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
       GPIO_PIN_RESET);
29  HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
       GPIO_PIN_SET);
30  HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
       GPIO_PIN_SET);
31
32  countdown(2);
33
34  // North-South RED, East-West GREEN
35  HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
       GPIO_PIN_RESET);
```

```
36    HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
          GPIO_PIN_SET);
37    HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
          GPIO_PIN_SET);
38
39    HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
          GPIO_PIN_SET);
40    HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
          GPIO_PIN_SET);
41    HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
          GPIO_PIN_RESET);
42
43    countdown(3);
44
45    // North-South RED, East-West YELLOW
46    HAL_GPIO_WritePin(NS_RED_GPIO_Port, NS_RED_Pin,
          GPIO_PIN_RESET);
47    HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port, NS_YELLOW_Pin,
          GPIO_PIN_SET);
48    HAL_GPIO_WritePin(NS_GREEN_GPIO_Port, NS_GREEN_Pin,
          GPIO_PIN_SET);
49
50    HAL_GPIO_WritePin(EW_RED_GPIO_Port, EW_RED_Pin,
          GPIO_PIN_SET);
51    HAL_GPIO_WritePin(EW_YELLOW_GPIO_Port, EW_YELLOW_Pin,
          GPIO_PIN_RESET);
52    HAL_GPIO_WritePin(EW_GREEN_GPIO_Port, EW_GREEN_Pin,
          GPIO_PIN_SET);
53
54    countdown(2);
55}
```

6 Exercise 6

6.1 Requirement

Exercise 6 introduces a new Proteus schematic that simulates an analog clock using twelve LEDs. The LEDs are arranged similarly to the twelve positions on a clock face and are connected to pins PA4 through PA15 of the STM32F103C6.

The pin assignments are:

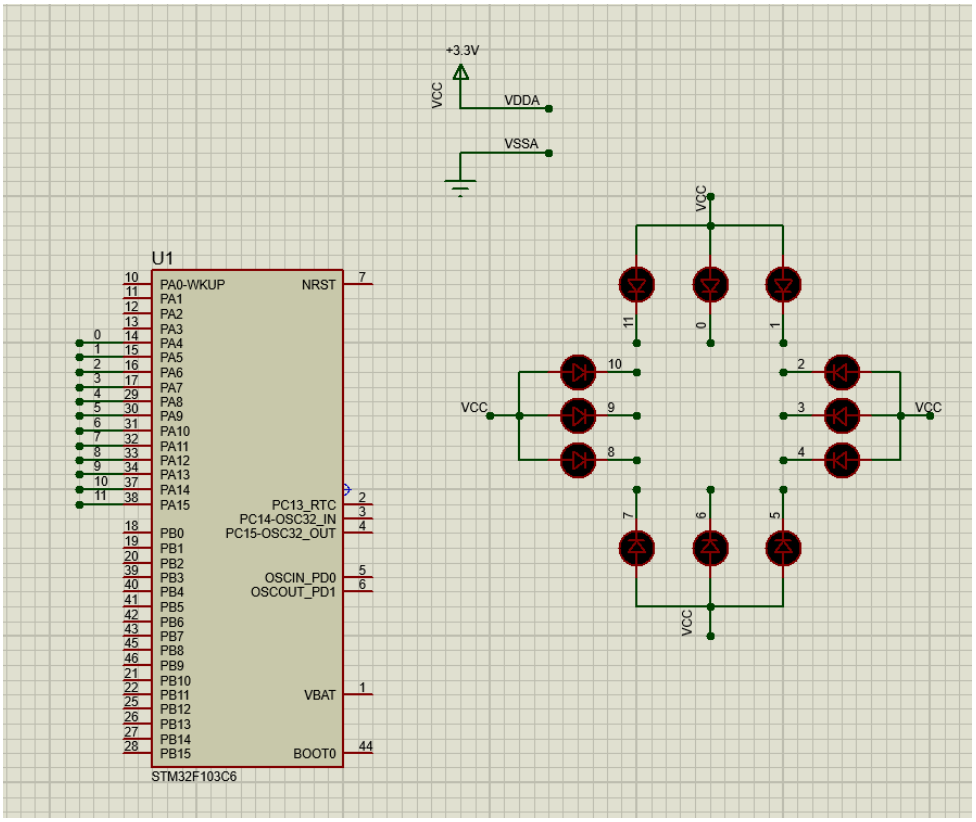
- LED 0: PA4.
- LED 1: PA5.
- LED 2: PA6.
- LED 3: PA7.
- LED 4: PA8.
- LED 5: PA9.
- LED 6: PA10.
- LED 7: PA11.
- LED 8: PA12.
- LED 9: PA13.
- LED 10: PA14.
- LED 11: PA15.

The LEDs are connected in an active-low configuration. Therefore, a logic-low output turns an LED on and a logic-high output turns it off.

A simple test program is implemented to turn on each LED sequentially in order to verify all twelve GPIO connections.

6.2 Circuit Schematic

Proteus schematic for the twelve-LED analog clock.



Source here

6.3 Source Code

```
static const uint16_t clock_pins[12] =
{
  GPIO_PIN_4,
  GPIO_PIN_5,
  GPIO_PIN_6,
  GPIO_PIN_7,
  GPIO_PIN_8,
  GPIO_PIN_9,
  GPIO_PIN_10,
  GPIO_PIN_11,
  GPIO_PIN_12,
```

```
  GPIO_PIN_13,
  GPIO_PIN_14,
  GPIO_PIN_15
};

// Turn all LEDs OFF
for (int i = 0; i < 12; i++)
{
  HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
}

while (1)
{
  for (int i = 0; i < 12; i++)
  {
    // Turn current LED ON
    HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_RESET);

    HAL_Delay(500);

    // Turn current LED OFF
    HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
  }
}
```

7 Exercise 7

7.1 Requirement

Exercise 7 implements the `clearAllClock()` function. The purpose of this function is to turn off all twelve LEDs of the analog clock.

Since the LEDs are connected in an active-low configuration, a logic-high output is used to turn each LED off.

7.2 Source Code

```
void clearAllClock(void)
{
  for (int i = 0; i < 12; i++)
  {
    HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
  }
}
```

8 Exercise 8

8.1 Requirement

Exercise 8 implements the `setNumberOnClock(int num)` function. The input value ranges from 0 to 11, where each value corresponds to one of the twelve LEDs of the analog clock.

The LED mapping follows the GPIO configuration from Exercise 6, with values 0 to 11 corresponding to pins PA4 to PA15.

Since the LEDs are connected in an active-low configuration, a logic-low output is used to turn the selected LED on.

8.2 Source Code

```
void setNumberOnClock(int num)
{
  if (num < 0 || num > 11)
    return;

  HAL_GPIO_WritePin(GPIOA, clock_pins[num], GPIO_PIN_RESET);
}
```

9 Exercise 9

9.1 Requirement

Exercise 9 implements the `clearNumberOnClock(int num)` function. The input value ranges from 0 to 11, where each value corresponds to one of the twelve LEDs of the analog clock.

The function turns off the LED associated with the specified number. The LED mapping follows the GPIO configuration from Exercise 6, with values 0 to 11 corresponding to pins PA4 to PA15.

Since the LEDs are connected in an active-low configuration, a logic-high output is used to turn the selected LED off.

9.2 Source Code

```
void clearNumberOnClock(int num)
{
  if (num < 0 || num > 11)
    return;

  HAL_GPIO_WritePin(GPIOA, clock_pins[num], GPIO_PIN_SET);
}
```

10 Exercise 10

10.1 Requirement

Exercise 10 integrates the previous functions to implement the complete twelve-LED analog clock. At a given time, the LEDs represent the hour, minute, and second positions.

The twelve LEDs represent the twelve positions of a clock face, where position 0 corresponds to 12 o'clock and positions 1 to 11 correspond to the remaining hour positions.

Since both minutes and seconds range from 0 to 59, their values are mapped to the twelve LED positions by dividing them by 5. The positions are therefore calculated as:

- Hour position: `hour % 12`.
- Minute position: `minute / 5`.



- Second position: `second / 5`.

The functions implemented in the previous exercises are reused to clear the clock and activate the required LEDs.

10.2 Source Code

```
void displayClock(int hour, int minute, int second)
{
  int hour_pos = hour % 12;
  int minute_pos = minute / 5;
  int second_pos = second / 5;

  clearAllClock();

  setNumberOnClock(hour_pos);
  setNumberOnClock(minute_pos);
  setNumberOnClock(second_pos);
}

int hour = 10;
int minute = 15;
int second = 30;

while (1)
{
displayClock(hour, minute, second);
HAL_Delay(1000);
second++;

if (second >= 60) {
  second = 0;
  minute++;

  if (minute >= 60) {
    minute = 0;
```

```
30      hour++;
31
32      if (hour >= 12){
33        hour = 0;
34      }
35    }
36 }
37 }
```

GitHub Repository

All Proteus schematics and related project files for Lab01 are available at: https://github.com/Kaytii210/Microprocessors-Microcontrollers/tree/main/Lab01